

# PROCESS DISPATCHER & CPU SCHEDULING ALGORITHMS

---

## *Group 10 Report*

CS3521 — Operating Systems

|                   |                                                                                     |
|-------------------|-------------------------------------------------------------------------------------|
| <b>Language</b>   | C (C99 standard)                                                                    |
| <b>Files</b>      | process.h, queue.h, queue.c,<br>scheduler.h, scheduler.c,<br>dispatcher.c, Makefile |
| <b>Algorithms</b> | FCFS, Round Robin, Shortest Job<br>First, Priority                                  |
| <b>Build</b>      | gcc -Wall -Wextra via Makefile                                                      |

## 1. Introduction

---

This report describes the design and implementation of a **CPU process dispatcher** and **four scheduling algorithms** written in the C programming language, as required by the CS3521 Operating Systems assignment. The objective of the assignment is to simulate the core mechanisms of a uniprocessor operating system scheduler: process arrival, queuing, dispatching, context switching, and completion tracking.

The implementation consists of six source files organised by responsibility: the Process Control Block definition, a linked-list ready queue, four scheduling algorithms, and a central dispatcher loop. A Makefile is included to build the project with a single command.

## 2. System Overview

The system is built from six source files. Each file has a clearly defined single responsibility. The diagram below shows how they depend on each other.

| File         | Type          | Responsibility                                     |
|--------------|---------------|----------------------------------------------------|
| process.h    | Header        | Defines the PCB struct — the core data structure   |
| queue.h      | Header        | Declares the Queue struct and its four operations  |
| queue.c      | Source        | Implements createQueue, enqueue, dequeue, isEmpty  |
| scheduler.h  | Header        | Declares the four scheduling algorithm functions   |
| scheduler.c  | Source        | Implements FCFS, Round Robin, SJF, Priority        |
| dispatcher.c | Source (main) | Main loop, simulation table, statistics output     |
| Makefile     | Build         | Compiles all three .c files with gcc -Wall -Wextra |

**process.h** is the foundation — every other file includes it either directly or transitively through queue.h. The dependency chain is: process.h → queue.h → (queue.c, scheduler.h) → (scheduler.c, dispatcher.c).

### 2.1 Build System — Makefile

The Makefile compiles all three C source files in one command and supports a clean target:

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -g
TARGET  = dispatcher
SRCS    = dispatcher.c queue.c scheduler.c
OBJS    = $(SRCS:.c=.o)

.PHONY: all run1 run2 run3 run4 runall clean

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

```
run1: $(TARGET)
    @echo "\n>>> Running FCFS <<<\n"
    ./$(TARGET) 1

run2: $(TARGET)
    @echo "\n>>> Running SJF <<<\n"
    ./$(TARGET) 2

run3: $(TARGET)
    @echo "\n>>> Running Round-Robin <<<\n"
    ./$(TARGET) 3

run4: $(TARGET)
    @echo "\n>>> Running Priority + RR <<<\n"
    ./$(TARGET) 4

runall: $(TARGET)
    ./$(TARGET) 1
    ./$(TARGET) 2
    ./$(TARGET) 3
    ./$(TARGET) 4

clean:
    rm -f $(TARGET) $(OBJS)
```

The `-Wall` `-Wextra` flags enable all standard warnings during compilation, ensuring the code is clean and free of implicit errors. Running `make` produces the executable `./dispatcher`, and `make clean` removes it.

## 3. Data Structures

---

### 3.1 The Process Control Block (PCB) — process.h

The PCB is the most fundamental data structure in the system. Every process that exists during simulation is represented by exactly one PCB allocated on the heap with `malloc()`. The struct is defined in `process.h` and included by every other module:

```
#ifndef PROCESS_H
#define PROCESS_H

typedef struct PCB {
    int pid;           /* unique process ID (1, 2, 3...) */
    int arrivalTime;   /* tick at which this process entered system */
    int maxTicks;      /* total CPU time needed to complete */
    int remainingTicks; /* counts down to zero - 0 means finished */
    int priority;      /* lower number = higher priority */
    int waitingTime;   /* total ticks spent waiting in the ready queue */
    int turnaroundTime; /* calculated at process completion */
    struct PCB *next;  /* pointer to next PCB - forms the linked list */
} PCB;

#endif
```

#### Field-by-Field Explanation

**pid** — A unique integer identifier assigned from the simulation table. Used in all output messages to identify which process is being discussed.

**arrivalTime** — Records the tick number at which the process was created and enqueued. Used at the end to calculate turnaround time.

**maxTicks** — The total number of CPU ticks the process needs to complete its work. Set once at creation and never changed. Used in statistics output.

**remainingTicks** — Starts equal to `maxTicks` and is decremented by 1 every tick the process runs on the CPU. When it reaches zero, the process is finished. This is the single most critical field — the entire scheduling simulation revolves around it.

**priority** — An integer where a lower value means higher urgency. For example, priority 1 will always be dispatched before priority 3 when both are in the queue. Used only by the Priority Scheduling algorithm.

**waitingTime** — Accumulated tick by tick during the simulation. Every tick that a process sits in the ready queue without running, this counter is incremented. Not touched for the currently running process.

**turnaroundTime** — Calculated at the exact moment the process finishes, using the formula:  $(\text{tick} + 1) - \text{arrivalTime}$ . This captures the total elapsed real time from arrival to completion, including both waiting and execution.

**\*next** — A pointer to the next PCB in the queue. This is what turns individual PCB structs into nodes of a singly linked list. No separate node struct is needed — the PCB itself acts as its own list node.

## 3.2 The Ready Queue — queue.h / queue.c

The ready queue is implemented as a **singly linked list** with two permanent pointers: `front` (the head — processes leave from here) and `rear` (the tail — processes arrive here). This design gives  $O(1)$  time complexity for both `enqueue()` and `dequeue()` without any array scanning.

```
/* queue.h */
typedef struct Queue {
    PCB *front; /* head of the list - next to be dispatched */
    PCB *rear; /* tail of the list - where new arrivals join */
} Queue;

Queue* createQueue();
void enqueue(Queue *q, PCB *process);
PCB* dequeue(Queue *q);
int isEmpty(Queue *q);
```

### **createQueue()**

Allocates a Queue struct on the heap with `malloc()` and initialises both pointers to NULL. Returns a pointer to the new queue. Called once at the start of `main()`.

### **enqueue(Queue \*q, PCB \*process)**

Adds a process to the back of the queue in  $O(1)$  time:

- Sets `process->next = NULL` to mark it as the new tail.
- If the queue is empty (`rear == NULL`), both `front` and `rear` point to the new node.
- Otherwise, chains the new node onto the current `rear` (`rear->next = process`) and advances the `rear` pointer.

```
void enqueue(Queue *q, PCB *process) {
    process->next = NULL;
    if (q->rear == NULL) {
        q->front = process;
        q->rear = process;
        return;
    }
    q->rear->next = process;
    q->rear = process;
}
```

### **dequeue(Queue \*q)**

Removes and returns the process at the front in  $O(1)$  time:

- Returns NULL immediately if the queue is empty.
- Saves the front pointer in a temporary variable.
- Advances `front` to `front->next`. If this makes `front` NULL (queue now empty), `rear` is also set to NULL.

- Returns the saved pointer.

```
PCB* dequeue(Queue *q) {
    if (q->front == NULL) return NULL;
    PCB *temp = q->front;
    q->front = q->front->next;
    if (q->front == NULL) q->rear = NULL;
    return temp;
}
```

### **isEmpty(Queue \*q)**

Returns 1 (true) if `q->front == NULL`, 0 otherwise. The queue being empty is fully determined by the front pointer — if it is NULL, there are no nodes.

## 4. Simulation Table and Process Creation

Rather than reading from an external file, the program uses a hard-coded `simEntry` table inside `dispatcher.c` to describe all processes. Each row specifies a process's ID, the tick it should arrive at, its total CPU time requirement, and its priority:

```
typedef struct {
    int id;
    int arriveAfterTick;
    int maxTicks;
    int priority;
} SimEntry;

SimEntry simulation[] = {
    {1, 0, 5, 2}, /* P1: arrives tick 0, needs 5 ticks, priority 2 */
    {2, 2, 3, 1}, /* P2: arrives tick 2, needs 3 ticks, priority 1 */
    {3, 4, 6, 3}, /* P3: arrives tick 4, needs 6 ticks, priority 3 */
    {4, 4, 2, 2}, /* P4: arrives tick 4, needs 2 ticks, priority 2 */
    {5, 6, 4, 1}, /* P5: arrives tick 6, needs 4 ticks, priority 1 */
};
int simSize = 5;
```

This table is **not** the process itself — it is raw configuration data. At the start of every tick, the dispatcher scans every row looking for entries where `arriveAfterTick == tick`. When a match is found, a brand new PCB is created dynamically:

```
for (int i = 0; i < simSize; i++) {
    if (simulation[i].arriveAfterTick == tick) {
        PCB *p = (PCB*)malloc(sizeof(PCB));
        p->pid = simulation[i].id;
        p->arrivalTime = tick;
        p->maxTicks = simulation[i].maxTicks;
        p->remainingTicks = simulation[i].maxTicks; /* starts full */
        p->priority = simulation[i].priority;
        p->waitingTime = 0;
        p->turnaroundTime = 0;
        p->next = NULL;
        enqueue(readyQueue, p);
    }
}
```

`malloc(sizeof(PCB))` allocates exactly enough memory on the heap to hold one PCB struct. The pointer `p` is then initialised field by field. Note that `remainingTicks` is always set equal to `maxTicks` — a new process has used none of its CPU time yet. Once fully initialised, the PCB is immediately placed at the back of the ready queue.

## 5. The Dispatcher — dispatcher.c

---

The dispatcher is the engine of the simulation. It is a `while` loop inside `main()` that runs until every process has finished (`doneCount == simSize`). Each iteration of the loop represents one **tick** of CPU time. Before the loop begins, the user is prompted to choose a scheduling algorithm and an optional real-time tick delay:

```
printf("Select scheduling algorithm:\n");
printf(" 1. FCFS      (First-Come-First-Served)\n");
printf(" 2. RR        (Round Robin, quantum=%d)\n", TIME_SLICE);
printf(" 3. SJF        (Shortest Job First)\n");
printf(" 4. Priority    (lower number = higher priority)\n");
scanf("%d", &choice);

printf("Tick speed in seconds (0 = instant): ");
scanf("%d", &tickDelay);
```

### 5.1 Per-Tick Execution Order

Every single tick executes the following five steps in strict order:

#### Step 1 — Print Tick Header

Prints the current tick number and the status of the CPU (idle or which process is running with how many ticks remaining). This provides a complete trace of the simulation.

#### Step 2 — Check for Arrivals

Scans the simulation table for any process whose `arriveAfterTick` equals the current tick. For each match, a new PCB is created with `malloc()`, initialised, and placed at the back of the ready queue with `enqueue()`. Multiple processes can arrive on the same tick (as P3 and P4 do at tick 4).

#### Step 3 — Scheduling Decision and Context Switch

This is where the selected algorithm is called. The current `runningProcess` pointer is saved in `prev` before the call. After the call, if `runningProcess != prev`, a context switch has occurred and it is printed. The mechanism is explained in detail in Section 6.

#### Step 4 — Update Waiting Times

A cursor pointer walks the entire ready queue and increments `waitingTime` on every PCB currently sitting in the queue. The running process is never in the queue during this step, so it is correctly excluded from the wait count.

```
PCB *cursor = readyQueue->front;
while (cursor != NULL) {
    cursor->waitingTime++;
    cursor = cursor->next;
}
```

### Step 5 — Execute One Tick and Check Completion

If a process is running, its `remainingTicks` is decremented by 1. If it reaches zero, the process has finished. The `turnaroundTime` is calculated as  $(\text{tick} + 1) - \text{arrivalTime}$ , the PCB pointer is stored in the `done[]` array, `runningProcess` is set to `NULL`, and `doneCount` is incremented. The `tick + 1` accounts for the fact that the process finishes at the end of the current tick.

```
if (runningProcess != NULL) {
    runningProcess->remainingTicks--;
    if (runningProcess->remainingTicks == 0) {
        runningProcess->turnaroundTime = tick + 1 - runningProcess->arrivalTime;
        done[doneCount++] = runningProcess;
        runningProcess = NULL;
        rrCounter = 0;
    }
}
```

## 6. Context Switching

---

In a real operating system, a context switch saves all CPU registers (program counter, stack pointer, general-purpose registers) from the current process into its PCB, then loads the next process's saved registers back into the CPU. This is why a preempted process can resume exactly where it left off.

In this simulation, the only meaningful state a process has is `remainingTicks` — how much CPU time it still needs. Because this value lives inside the PCB struct itself (which is heap-allocated and never moves), saving and restoring state is automatic. The context switch therefore reduces to a single pointer reassignment:

```
PCB *prev = runningProcess;          /* remember who was running */

/* --- call scheduling algorithm --- */
runningProcess = FCFS(readyQueue);   /* or RoundRobin, SJF, Priority */

/* if runningProcess != prev, a context switch just happened */
if (runningProcess != NULL && runningProcess != prev)
    printf(" -> Context switch: P%d dispatched\n", runningProcess->pid);
```

When a process is **preempted** (specifically in Round Robin), it is passed back to `enqueue()` with its current `remainingTicks` value unchanged. Because `enqueue()` takes a pointer to the existing heap-allocated PCB (not a copy), the state is fully preserved in memory and will be available when the process is dequeued again later. This is the C-level equivalent of saving context to the PCB.

### Key insight:

Only the `runningProcess` pointer moves. Both PCBs (old and new) remain intact on the heap throughout. The preempted process's `remainingTicks` value is preserved because it lives inside the PCB struct, not in any CPU register or local variable.

## 7. Scheduling Algorithms — scheduler.c

All four algorithms are implemented in `scheduler.c` and declared in `scheduler.h`. They all receive a pointer to the ready queue and return a pointer to the PCB that should run next. The dispatcher loop treats them as interchangeable — only Step 3 of each tick changes between algorithms.

| Algorithm          | Selection Method                    | Preemptive? | Context Switch When            |
|--------------------|-------------------------------------|-------------|--------------------------------|
| FCFS               | Front of queue (arrival order)      | No          | Process finishes               |
| Round Robin        | Front of queue, cyclic with quantum | Yes         | Time slice expires OR finishes |
| Shortest Job First | Minimum remaining Ticks in queue    | No          | Process finishes               |
| Priority           | Minimum priority number in queue    | No          | Process finishes               |

### 7.1 First-Come-First-Served (FCFS)

FCFS is the simplest scheduling algorithm. It dispatches processes in the order they arrive — whoever has been waiting longest goes next. It is **non-preemptive**: once a process starts running, it continues until it finishes with no interruptions.

```
PCB* FCFS(Queue *readyQueue) {
    if (!isEmpty(readyQueue)) {
        return dequeue(readyQueue); /* take from the front — O(1) */
    }
    return NULL;
}
```

In the dispatcher, FCFS is only called when `runningProcess == NULL`, enforcing non-preemption: whatever is dispatched keeps running until it finishes. It simply calls `dequeue()`, which pops the front of the linked list — the process that arrived first. This is  $O(1)$ .

**Strength:** Simple, fair in order of arrival, no starvation.

**Weakness:** The convoy effect — if a long process arrives first, all shorter processes behind it are blocked until it finishes, producing high average waiting times.

### 7.2 Round Robin (RR)

Round Robin is the only **preemptive** algorithm in the implementation. Each process is given a fixed time quantum (`TIME_SLICE = 2` ticks). If a process has not finished when its quantum

expires, it is preempted and placed at the back of the queue. The next process at the front is then dispatched.

An `rrCounter` variable (passed by pointer so the function can modify it) tracks how many consecutive ticks the current process has run this quantum.

```
PCB* RoundRobin(Queue *readyQueue, PCB *runningProcess,
                int timeSlice, int *timeCounter) {
    if (runningProcess != NULL) {
        (*timeCounter)++;
        if (*timeCounter >= timeSlice) {
            *timeCounter = 0;
            enqueue(readyQueue, runningProcess); /* re-queue with state preserved */
            return dequeue(readyQueue);          /* pick next in line */
        }
        return runningProcess; /* still within slice - keep running */
    }
    *timeCounter = 0;
    return dequeue(readyQueue); /* CPU was idle - start fresh */
}
```

Unlike the other three algorithms, RoundRobin is called **every tick** regardless of whether a process is running. When the slice expires, `enqueue(readyQueue, runningProcess)` puts the current process back at the tail of the queue — its `remainingTicks` value is already correct because it has been decremented each tick. This is the context-save operation. `dequeue()` then returns the next process from the head.

**Strength:** Fair to all processes. No process waits longer than  $(n-1) * \text{timeSlice}$  ticks before getting another turn. Excellent for interactive systems.

**Weakness:** Higher context switch overhead. A very small quantum increases switches; a large quantum degrades toward FCFS behaviour.

### 7.3 Shortest Job First (SJF)

SJF always dispatches the process with the fewest remaining ticks. It is **non-preemptive**: once a process starts, it runs to completion. This algorithm minimises average waiting time when all processes are available simultaneously.

Because the ready queue is a linked list (not a sorted structure), SJF must scan the entire queue every time it selects a process. After finding the shortest job, it surgically removes it by re-linking the `prev->next` pointer around it:

```
PCB* ShortestJobFirst(Queue *readyQueue) {
    if (isEmpty(readyQueue)) return NULL;

    /* Pass 1: find the node with fewest remainingTicks */
    PCB *shortest = readyQueue->front;
    PCB *current = readyQueue->front->next;
    while (current != NULL) {
```

```
        if (current->remainingTicks < shortest->remainingTicks)
            shortest = current;
        current = current->next;
    }

    /* Pass 2: remove the chosen node from the linked list */
    PCB *prev = NULL;
    current = readyQueue->front;
    while (current != shortest) {
        prev = current;
        current = current->next;
    }
    if (prev == NULL) readyQueue->front = shortest->next;
    else prev->next = shortest->next;
    if (readyQueue->rear == shortest) readyQueue->rear = prev;
    shortest->next = NULL;
    return shortest;
}
```

**Strength:** Provably optimal for minimising average waiting time when all processes are present at once.

**Weakness:** Starvation — if short jobs keep arriving continuously, long jobs may never be dispatched.

## 7.4 Priority Scheduling

Priority scheduling selects the process with the lowest priority number (priority 1 is higher urgency than priority 3). It is structurally identical to SJF — the same two-pass scan-and-remove pattern — but compares `priority` instead of `remainingTicks`:

```
PCB *PriorityRR(Queue *readyQueue, PCB *runningProcess, int timeSlice) {

    if (isEmpty(readyQueue)) {
        /* Nothing in queue - keep running the current process if any */
        return runningProcess;
    }

    /* Find the best (lowest) priority value in the ready queue */
    int bestPriority = readyQueue->front->priority;
    PCB *cur = readyQueue->front->next;
    while (cur) {
        if (cur->priority < bestPriority)
            bestPriority = cur->priority;
        cur = cur->next;
    }

    /* If something is running, compare its priority */
    if (runningProcess != NULL) {
        if (runningProcess->priority < bestPriority) {
            /* Running process has strictly higher priority - keep running */
            return runningProcess;
        }
        if (runningProcess->priority == bestPriority) {
            /* Same priority level - apply RR */

```

```
        if (runningProcess->rrCounter < timeSlice) {
            return runningProcess;    /* Still within its slice */
        }
        /* Slice expired - preempt and re-enqueue */
        runningProcess->rrCounter = 0;
        enqueue(readyQueue, runningProcess);
        runningProcess = NULL;
    } else {
        /* A higher-priority process has arrived - preempt immediately */
        runningProcess->rrCounter = 0;
        enqueue(readyQueue, runningProcess);
        runningProcess = NULL;
    }
}

/* Pick the highest-priority process from the queue.
 * Among equals keep FIFO order (first one found wins). */
PCB *chosen      = NULL;
PCB *chosenPrev  = NULL;
cur              = readyQueue->front;
PCB *prev        = NULL;

while (cur) {
    if (chosen == NULL || cur->priority < chosen->priority) {
        chosen      = cur;
        chosenPrev  = prev;
    }
    prev = cur;
    cur  = cur->next;
}

/* Unlink chosen */
if (chosenPrev == NULL) {
    readyQueue->front = chosen->next;
} else {
    chosenPrev->next = chosen->next;
}
if (chosen == readyQueue->rear) {
    readyQueue->rear = chosenPrev;
}
chosen->next = NULL;
readyQueue->size--;

return chosen;
}
```

In the simulation table, priorities are defined as: P1=2, P2=1, P3=3, P4=2, P5=1. This means P2 and P5 (priority 1) will always be chosen over P1 and P4 (priority 2), which will always be chosen over P3 (priority 3). And for processes with the same priority, the round robin scheduling algorithm is used for tie breaking between such processes.

**Strength:** Critical tasks (lower priority number) always run before less important ones.

**Weakness:** Low-priority processes can starve indefinitely if high-priority processes keep arriving.

## 8. Statistics Output

When all processes finish, the `printStats()` function is called with the `done[]` array (an array of PCB pointers accumulated as processes finished in order). It prints a formatted per-process table and computes averages:

```
void printStats(PCB **done, int count, int totalTicks) {
    printf("%-5s %-8s %-8s %-10s %-12s\n",
           "PID", "Arrival", "Burst", "Turnaround", "Waiting");
    double avgT = 0, avgW = 0;
    for (int i = 0; i < count; i++) {
        printf("%-5d %-8d %-8d %-10d %-12d\n",
               done[i]->pid, done[i]->arrivalTime, done[i]->maxTicks,
               done[i]->turnaroundTime, done[i]->waitingTime);
        avgT += done[i]->turnaroundTime;
        avgW += done[i]->waitingTime;
    }
    printf("Average turnaround: %.2f Average waiting: %.2f\n",
           avgT / count, avgW / count);
}
```

### 8.1 Metric Definitions

**Waiting Time** — The total number of ticks a process spent in the ready queue without running. Accumulated live during the simulation: every tick that a process is in the queue, its `waitingTime` is incremented.

**Turnaround Time** — The total elapsed time from the tick a process arrived to the tick it finished. Formula:  $(\text{tick} + 1) - \text{arrivalTime}$ . Includes both waiting time and execution time.

**Average Waiting Time** — Sum of all waiting times divided by the number of processes. Lower is better; SJF typically produces the lowest value.

**Average Turnaround Time** — Sum of all turnaround times divided by the number of processes. A key measure of overall system responsiveness.

### 8.2 Sample Output — FCFS

Running the simulation with FCFS produces the following output:

```
===== STATISTICS =====
PID   Arrival  Burst   Turnaround  Waiting
-----
1      0         5        5           0
2      2         3        8           5
3      4         6       16          10
4      4         2       18          16
5      6         4       19          15
-----
Average turnaround time : 13.20 ticks
Average waiting time    : 9.20 ticks
```

```
Total ticks elapsed      : 20
```

P1 has zero waiting time because it arrives at tick 0 and is immediately dispatched. P2 waits from tick 2 until tick 5 (when P1 finishes), giving it a waiting time of 3 ticks. Each subsequent process waits longer because it must wait for all earlier processes to complete — this is the **convoy effect** inherent in FCFS when burst times vary.

## 9. Memory Management

---

All PCBs are dynamically allocated on the heap with `malloc()` when a process arrives. They are **not freed immediately** when a process finishes — instead, the pointer is stored in the `done[]` array so that statistics can be printed after the loop ends. After `printStats()` is called, all PCBs are freed in a final loop:

```
for (int i = 0; i < doneCount; i++) free(done[i]);
free(readyQueue);
```

The Queue struct itself is also freed. No memory leaks occur in the normal execution path. Any PCB still in the ready queue at the end of the loop (which should not happen since the loop exits only when all processes finish) would represent a leak, but the loop condition guarantees this cannot occur.

## 10. Design Decisions and Discussion

---

### 10.1 PCB as Linked List Node

Rather than creating a separate `Node` struct to wrap the PCB, the `*next` pointer is embedded directly in the PCB struct. This reduces memory allocation overhead (one `malloc()` per process instead of two) and simplifies pointer management. The trade-off is that a PCB can only exist in one linked list at a time — which is acceptable because a process is always either in the ready queue or running, never both.

### 10.2 Non-Preemptive vs Preemptive Design

FCFS, SJF, and Priority are non-preemptive: they only make a scheduling decision when the CPU is free (`runningProcess == NULL`). Round Robin is preemptive: it is called every tick and can interrupt the current process. This difference is enforced in the dispatcher's Step 3 with an `if` guard around the non-preemptive algorithms.

### 10.3 Shared Node-Removal Logic in SJF and Priority

SJF and Priority share the identical two-pass pattern: scan the list to find the best candidate, then re-link pointers to remove it. The only difference is the comparison field (`remainingTicks` vs `priority`). This demonstrates that the data structure and the scheduling policy are cleanly separated — a different policy requires only a different comparison, not a different algorithm structure.

### 10.4 Real-Time Tick Delay

The:

```
static struct timespec simStart;
static void startClock(void) {
    clock_gettime(CLOCK_MONOTONIC, &simStart);
}
...
static double elapsedSeconds(void) {
    struct timespec now;
    clock_gettime(CLOCK_MONOTONIC, &now);
    return (now.tv_sec - simStart.tv_sec) +
        (now.tv_nsec - simStart.tv_nsec) / 1e9;
}
...
startClock();
```

Were added to make the simulation observable in real time. Each tick takes exactly one millisecond, making it easy to watch processes arrive, run, and finish at human speed.

## 11. Conclusion

---

This implementation fulfils all requirements of the CS3521 assignment and goes beyond the minimum by providing all four scheduling algorithms, a runtime selection menu, real-time tick delay, and a comprehensive statistics output.

The system is structured around three core components: the **PCB** (a self-linking struct that doubles as a linked list node), the **Queue** (a singly linked list with  $O(1)$  enqueue and dequeue), and the **dispatcher loop** (a five-step per-tick cycle that handles arrivals, scheduling decisions, waiting time accumulation, execution, and completion detection).

Context switching is implemented as a single pointer reassignment. Because all process state lives inside the heap-allocated PCB, re-queuing a preempted process automatically preserves its state without any explicit save/restore operation.

The four algorithms demonstrate the key spectrum of scheduling strategies: FCFS prioritises fairness by arrival order, Round Robin distributes CPU time equitably with a fixed quantum, SJF minimises average waiting time by always choosing the shortest available job, and Priority ensures important processes are never delayed by less critical ones.